



AePS API SPECIFICATION

Table of Content

- 1 Business Partner Onboarding (Sales Representative will help in onboarding the partner and generating the keys for APIs).
- 2 Authorization and Authentication.
- 3 Bank List.
- 4 Bank IIN List.
- 5 State List.
- 6 District List.
- 7 Merchant Registration.
- 8 Send OTP.
- 9 Resend OTP.
- 10 OTP Verification.
- 11 E-KYC.
- 12 2 Factor Authentication.

Table of Content

- 12 AePS Txn OTP.
- 13 AePS Transaction.
- 14 API Specification Status Code.
- 15 Sample Code for Encryption and Decryption.
- 16 Escalation Matrix.

Sandbox Environment Details

Environment	Base URL
Sandbox	http://gateway.vimopay.in
Production	After audit team will share

Sandbox Credentials	
secretKey	f37701ee0778cfe1dd97b7537ae71709
saltKey	bb13602c407b1ddd6506d59e410bafeb
encryptdecryptKey	47cdba1911f078afe863774a8dffcb26
userId	E5B82667-9A9D-4A5A-A55C-F3B1E10BF370

Note: In UAT Environment pipe value will be 1.

For Production, team will share after the audit is done.

Eg. For Bank 1, pipe's value would be 1. For Bank 2, pipe's value would be 2, and so on...

Transaction Codes

Service Type	Code
Cash Withdrawal	CW
Balance Enquiry	BE
Mini Statement	MS
Aadhaar Pay	AP
Cash Deposit	CD

Gender Codes

Gender Code	Code
Male	M
Female	F
Others	O

Authentication Codes

Service Type	Code
Biometric Authentication	BA
Face Authentication	FA

Account Type

Account Type
Savings Account
Current Account

Dual Authentication Codes

Service Type	Code
Cash Withdrawal	CWTFA
Aadhaar Pay	APTFA

MCC Codes

Service Type	Code
4215	Courier services — air and ground and freight forwarders
4722	Travel agencies and tour operators
4812	Telecommunication equipment and telephone sales
4814	Telecommunication services, including local and long distance calls
4900	Utilities — electric, gas, water and sanitary
5044	Office, photographic, photocopy and microfilm equipment
5045	Computers, computer peripheral equipment — not elsewhere
5111	Stationery, office supplies and printing and writing paper
5331	Variety Stores
5411	Groceries and supermarkets
6533	Payment Transaction Provider- Merchant-Payment Transaction

Partner Authorization

- **URL** - /aepsapi/api/signature/authorizeuat
- **Method Type** - HTTP POST
- **Description** - Authorization API is used to check the request coming to the Vidual gateway is from the authorized partner / Merchant or not.

Headers Parameter	Type	Data Type	Description	Requisite
secretKey	Authorization	Alphanumeric	Secret Key partner will get at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory
saltKey	Authorization	Alphanumeric	Salt Key partner will get at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory
encryptdecryptKey	Authorization	Alphanumeric	Encrypt Decrypt Key partner will get at the time of partner's onboarding. This key is used for authorization purpose as well as to encrypt and decrypt the payload.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

• Postman Sample Request

Params	Authorization	Headers (11)	Body	Scripts	Tests	Settings	Cookies
☒	Connection	☐	keep-alive				
☒	secretKey		*****				
☒	saltKey		*****				
☒	encryptdecryptKey		*****				
☒	userId		*****				
	Key		Value				Description

• Sample Response

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data":  
    "7xb0GEfOmifv4lXOZa2TdS8Y2Q41slvS++I7NdFgyN4aNlVAzIZ+TKDcanuR1x1U4KT9Bg2c8p3RY5nyeOj2zkGFa7JcAKtQcrjhUETasS93B9HLr  
uXhVVr1gAZQoNFz0vQkC/WL2oKqt+prY6O+t8shtVI32/2Q76JueMprxUhZsSulWUCWpr2e6WzSfsiJryouQHBeWTf4E6tld8ZFz2QMT+PcJiq6  
3Yu3WSsAg2AlTg8MHBIZLYileELn3DgqKpakE4JQEYYUHpmom+t8ac9XMvFLfCk3iR4dc/Pa+0dtLXc0MVLtNXD/DIZuJGxv11F8fY0SrfP0dvfV  
H+LVKh5/2/N8CHyP+GZB2j7sDSptufcOMpSix7WYjZLtX2PPPlRvyukFpSTXHCIXu1+yR4XnS2FisEwYTOZ5sGtVF5FR+0JVj/Qq433BloXRXDa  
wVbbBqxkOWTdbg4UhFof8wYdm9dw6QT6S5WZKAuQjuWq+D6ynS2EBCj5iXX8tkoFnNhZPdZcfwV/7CRV8kzBHo+JCaPihXQMAXferwok9C  
/bdDH1ITipEZB/MHy1zModrhBO8P5zXZL8rXHMD"  
}
```

Bank List API

- **URL** - masterapi/api/master/banklistuat
- **Method Type** - HTTP GET
- **Description** - This API is used to get bank list that can be used to transfer the money to the beneficiary bank account using payout API.
- **Authorization Type** - Bearer Token (token value which we received from the partner authorization API call need to pass here.)

Headers Parameter	Type	Data Type	Description	Requisite
BearerToken	Authorization	Alphanumeric	token value which you received from the partner authorization API.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

• Postman Request to pass Header

Params	Authorization	Headers (10)	Body ●	Scripts	Tests	Settings	Cookies
☒	User-Agent	☐	PostmanRuntime/7.40.0				
☒	Accept	☐	*/*				
☒	Accept-Encoding	☐	gzip, deflate, br				
☒	Connection	☐	keep-alive				
☒	userId		*****				
☒	Authorization		Bearer KL2yN7QEeBGfzF/iuoHZV3xz9Z8YXD...				
	Key		Value				Description

- **Sample Response with Encrypted Data**

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data":  
    "PTPnWGHcVnp/YU0GX0yACWR5XWhDaxeRuS/j2Zga/yJGij+h88yDeK6b3ZzB1LzxPgj60DPoLXVpQTP4alc1WA2WAFySRTngmW0Fb  
t  
hIF/IK2MG2r/AG+4I/sbeSI98WBkq9XjAvGha0hRocZA6PbHN/xRJDWXdVtLDPdyRt8s1Ifua0hQKJBF+cxe1MWV/OqwUEzcggen7uYE+2  
hXsEa4QaLfE+zWRyFEJJGqslbxmVbeFAnx6EGkbbAr0/VEQUC3/oi19ZwZ7IB"  
}
```

• Sample Response with Decrypted Data

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data": [  
    {  
      "description": "Axis Bank",  
      "code": "001"  
    },  
    {  
      "description": "Bandhan Bank",  
      "code": "002"  
    }  
  ]  
}
```

Bank IIN List

- **URL** - /aepsapi/api/payment/bankiinuat
- **Method Type** - HTTP POST
- **Description** - This API is used to get Bank IIN list to initiate the AePS Transaction.
- **Authorization Type** - Bearer Token (token value which we received from the partner authorization API call need to pass here.)

Headers Parameter	Type	Data Type	Description	Requisite
BearerToken	Authorization	Alphanumeric	token value which you received from the partner authorization API.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
txnCode	Body Param	Alphabetic	User needs to define type of Transaction. Eg. - Cash Withdrawl(CW) - Cash Deposit (CD), - Balance Enquiry(BE), etc	Mandatory
authType	Body Param	Alphabetic	Type of authentication.	Mandatory

• Postman Request to pass Header

Params	Authorization	Headers (10)	Body ●	Scripts	Tests	Settings	Cookies
☒	User-Agent	☐	PostmanRuntime/7.40.0				
☒	Accept	☐	*/*				
☒	Accept-Encoding	☐	gzip, deflate, br				
☒	Connection	☐	keep-alive				
☒	userId		*****				
☒	Authorization		Bearer KL2yN7QEeBGfzF/iuoHZV3xz9Z8YXD...				
	Key		Value				Description

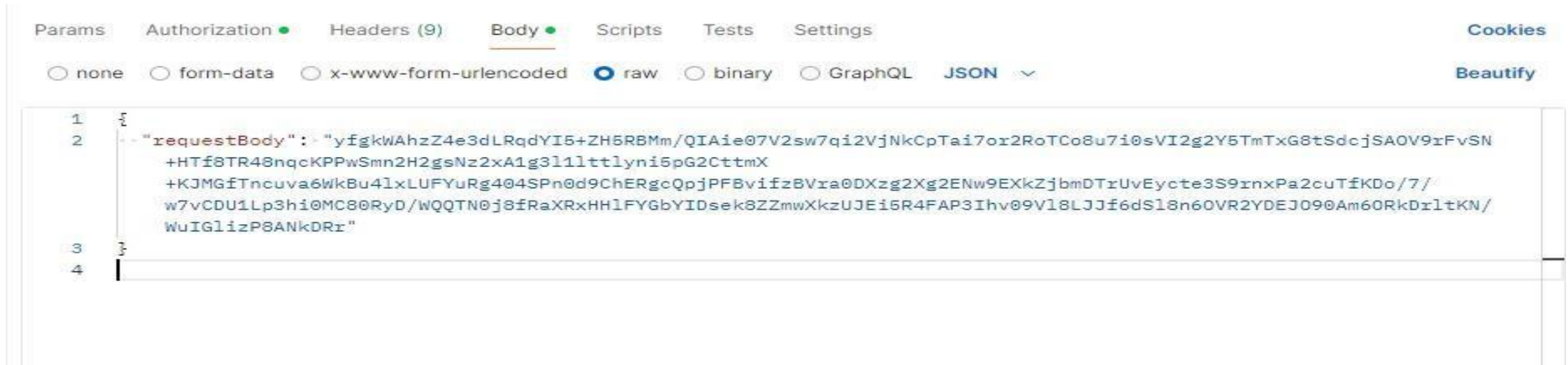
- Sample Plain Text Payload

```
{  
  "txnCode": "BE",  
  "authType": "BA"  
}
```

• Sample Plain Text Payload

```
{  
  "requestBody":  
  "Gll4gsT8Hf+xV8PofIIh6PQ30AdF5ldYZeDPqivup4783W8M5rSH5Juh2y2hF7+Dyv/His4JVU6ZZ6haYTml+Bksucfa4+lKCHnePw1rabf8  
  qHlCfMzP0duODq6sz5EVlwPBLqva+9JrVtcmrAvq/YEcXPTTUs9pisu5BN2BVWy8ElNu4t0YPoyqMRdzVmMh"  
}
```

Postman Request to pass Body Parameter



• Sample Response with Encrypted Data

```
{
  "successStatus": true,
  "message": "Success",
  "responseCode": "000",
  "data":
    "1aHQwl94wcBE1ISfbx9RwF0erJS7rDrMx+3B0oHzVj4a5lsbhwcScd8T4D2GodkBti858BhRDSB5v5rKll7KuS6xmOaXTXD0mqUgu/dwQKN
    WYSniD9x9XcSRAVZeoVE0AtMgxXJ8Bm36VwqXoCCbhi7Zocl4Eeib2M99UJm+Np0/+H6Vnt9gzW524391nOTn3laUf46lu5riS3Htmwa1gY
    K1nNvL4jTuAaM5GQTCzgWCyzlDgvrZk7k3fFPsJ9+fzAgocJQ364UcnhxbV+m8vUORC0ojUGJzBOQi9tdDuC+Jgl8ZSZwztk7Gyd9gCZV4ih
    ZsNlatoQVns7G5QkboGe47WN+fiMyxS2kzZorGWgRNU7xhV94BYjObQxO9/mSlrHd/WT1uPolSyXhy6sj3s4GW/lxguZ0nFyihDG8ZNRKna
    oY1Wz91d5Tc7fyeWpJiRI9ZohoEgs0rQk/r6zzRty2lEwe6EokKHfdYGOnccqeVldQONCjRvhCJriJjpiEhsGPAdBB43Zil5/XJA7pPeber1S5P
    G9cOcGG/jWyQReUGg639Z/i4mccpAqRZP9vekOjnaceapzB84e/6VqX929le2lxCzJiHLrIwWLBPHo4h5xSFlq9/Q+ARzzLiyCdNIGqt+ihlWi
    UUWbj0HQJaUKSt/28aV6fxlsOBFluQDXewfljUCy6BJjtT964s05WCWdj4RmcJWXJiAhTX7rJbWEDUIDCJc3Shrli9OKC1Qj/J4W0Mk7Ywc
    ptll62rp74479odNVFfKUUNHpBxbHa7dfBUEP6J9KzTjP1zZ2pNZyVMWPgatVSLVrFlRDMmDd5PIW0nxzunjlBYSDVKq7oTI64Eii4K9dPb3v
    L/o581qpYKCKQKD8ina3Rls3mS+QSBh4Mk9ln708P6RbM1KjoFmO6o8hYySjQPb+BsbyhpjHg0Dt6Sdcz1fhYdG"
}
```

Sample Response with Decrypted Data

```
[  
  {  
    "iin": "606985",  
    "description": "Bank Of Baroda"  
  },  
  {  
    "iin": "607140",  
    "description": "Himachal Pradesh Gramin Bank"  
  }  
]
```

State List

- **URL** - /masterapi/api/master/statelistuat
- **Method Type** - HTTP GET
- **Description** - This API is used to get state list for merchant's state from where the AePS Transaction is being initiated.
- **Authorization Type** - Bearer Token (token value which we received from the partner authorization API call need to pass here.)

Headers Parameter	Type	Data Type	Description	Requisite
Bearer Token	Authorization	Alphanumeric	token value which you received from the partner authorization API.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

• Postman Request to pass Header

Params	Authorization	Headers (10)	Body ●	Scripts	Tests	Settings	Cookies
☒	User-Agent	☐	PostmanRuntime/7.40.0				
☒	Accept	☐	*/*				
☒	Accept-Encoding	☐	gzip, deflate, br				
☒	Connection	☐	keep-alive				
☒	userId		*****				
☒	Authorization		Bearer KL2yN7QEeBGfzF/iuoHZV3xz9Z8YXD...				
	Key		Value				Description

- Sample Response with Encrypted Data

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data": "yfgjThx0TJWuM6xyLL9/8ZX2bRgK+A8ewv0nZmsmkP6xDjl/aMuA2+EW1ilvHala6D1NWJeB6  
bAlqh1N2+GGbBqUAgd8leDF+xi/wARe8myJEZTQUj3+GQBU CXCKQXNunG1RsBKjndpJyhNAw  
ELOaeyJPmEnvrrK6zCpgEGff+7TrNYbS+Cia4eLEhYAab2/PTPnWGhCvNp/YU0GX0yACWR5XW  
hDaxeRuS/j2Zga/yJGij+h88yDeK6b3ZzB1LzxPgj60DPoLXVpQTP4alc1WA2WAFySRTngmW0Fbt  
JBhIF/IK2MG2r/AG+4I/sbeSI98WBkq9XjAvGha0hRocZA6PbHN/xRJDW XDVTLDpdyRt8s1Ifua0hQKJBF+cxe1MWV/OqwUEzcggen7uYE  
+2hXsEa4QaLfE+zWRyFEJJGqslbxmVbeFAnx6EGkbbAr0/VEQUC3/oi19ZwZ7IB"  
}
```

- **Sample Response with Decrypted Data**

```
{
  "successStatus": true,
  "message": "Success",
  "responseCode": "000",
  "data": [
    {
      "description": "Andra Pradesh",
      "code": "AP "
    },
    {
      "description": "Assam",
      "code": "AS "
    }
  ]
}
```

District List

- **URL** - /aepsapi/api/payment/acquiredistrictuat
- **Method Type** - HTTP POST
- **Description** - This API is used to get district list for merchant's district from where the AePS Transaction is being initiated.
- **Authorization Type** - Bearer Token (token value which we received from the partner authorization API call need to pass here.)

Headers Parameter	Type	Data Type	Description	Requisite
BearerToken	Authorization	Alphanumeric	token value which you received from the partner authorization API.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
stateCode	Body Param	Alphabetic	Code of the state which the merchant can fetch from the State List API.	Mandatory

- Postman Request to pass Header

Params	Authorization	Headers (10)	Body ●	Scripts	Tests	Settings	Cookies
☒	User-Agent	ⓘ	PostmanRuntime/7.40.0				
☒	Accept	ⓘ	*/*				
☒	Accept-Encoding	ⓘ	gzip, deflate, br				
☒	Connection	ⓘ	keep-alive				
☒	userId		*****				
☒	Authorization		Bearer KL2yN7QEeBGfzF/iuoHZV3xz9Z8YXD...				
	Key		Value				Description

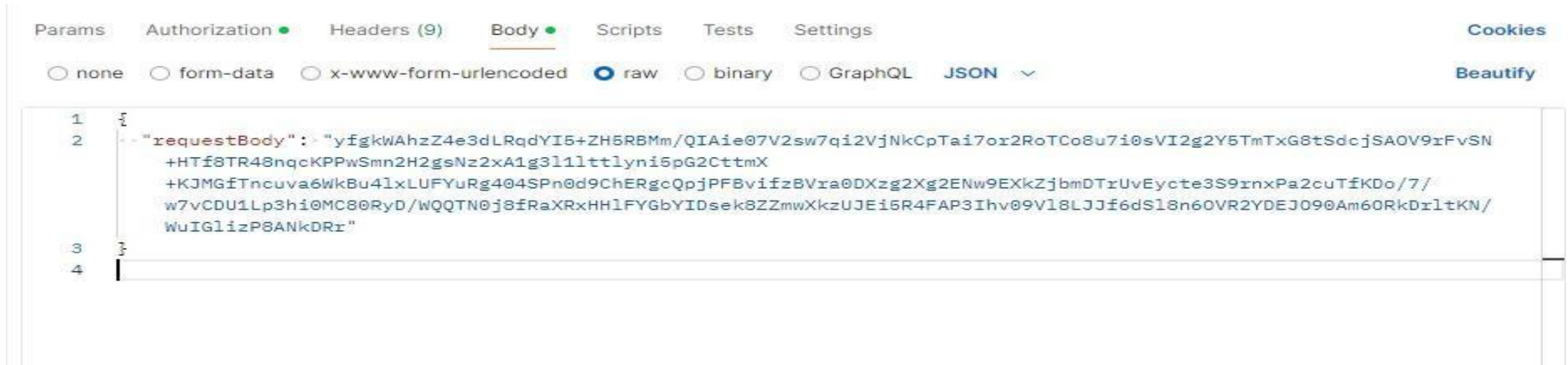
- Sample Plain Text Payload

```
{  
  "stateCode": "DL"  
}
```

• Sample Plain Text Payload

```
{
  "requestBody":
  "Gll4gsT8Hf+xV8PofIIh6PQ30AdF5ldYZeDPqivup4783W8M5rSH5Juh2y2hF7+Dyv/His4JVU6ZZ6haYTml+Bksucfa4+lKCHnePw1rabf8
  qHlCfMzP0duODq6sz5EVlwPBLqva+9JrVtcmrAvq/YEcXPTTUs9pisu5BN2BVWy8ElNu4t0YPoyqMRdzVmMh"
}
```

Postman Request to pass Body Parameter



- Sample Response with Encrypted Data

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data": "yfgjThx0TJWuM6xyLL9/8ZX2bRgK+A8ewv0nZmsmkP6xDjl/aMuA2+EW1ilvHala6D1NWJeB6  
bAlqh1N2+GGbBqUAgd8leDF+xi/wARe8myJEZTQUj3+GQBUcXCKQXNunG1RsBKjndpJyhNAw  
ELOaeyJPmEnvrrK6zCpgEGff+7TrNYbS+Cia4eLEhYAab2/PTPnWGhCvNp/YU0GX0yACWR5XW  
hDaxeRuS/j2Zga/yJGij+h88yDeK6b3ZzB1LzxPgj60DPoLXVpQTP4alc1WA2WAFySRTngmW0Fbt  
JBhIF/IK2MG2r/AG+4I/sbeSI98WBkq9XjAvGha0hRocZA6PbHN/xRJDWXdVtLDPdyRt8s1Ifua0hQKJBF+cxe1MWV/OqwUEzcggen7uYE  
+2hXsEa4QaLfE+zWRyFEJJGqslbxmVbeFAnx6EGkbbAr0/VEQUC3/oi19ZwZ7IB"  
}
```

- Sample Response with Decrypted Data

```
[  
  {  
    "code": "MAY",  
    "description": "Mayur Vihar"  
  },  
  {  
    "code": "SHA",  
    "description": "Shahdara"  
  },  
  {  
    "code": "DWK",  
    "description": "Dwarka"  
  }  
]
```

Merchant Registration

- **URL** - /aepsapi/api/payment/merchantonboarduat
- **Method Type** - HTTP POST
- **Description** - This API is used to onboard a merchant to initiate an AePS Transactions. For each request merchant has to pass a unique merchant ref Id. Merchant must be active to initiate any financial or non-financial transaction.
- **Authorization Type** - Bearer Token (token value which we received from the partner authorization API call need to pass here.)

Headers Parameter	Type	Data Type	Description	Requisite
BearerToken	Authorization	Alphanumeric	token value which you received from the partner authorization API.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
firstName	Body Param	Alphabetic	First name of the merchant	Mandatory
middleName	Body Param	Alphabetic	Middle name of the merchant	Non-Mandatory
lastName	Body Param	Alphabetic	Last name of the merchant	Mandatory
dob	Body Param	Numeric	Date of Birth of the merchant. Format will be in DD-MM-YYYY	Mandatory
emailId	Body Param	Alphanumeric	Email of the merchant.	Mandatory
merchantPhoneNumber	Body Param	Numeric	Mobile Number of the merchant.	Mandatory
shopAddress	Body Param	Alphanumeric	Address of the merchant's shop	Mandatory
merchantPinCode	Body Param	Numeric	Pin Code of the Merchant's residence	Mandatory
shopPan	Body Param	Alphanumeric	Pan Card Number of the Shop	Mandatory
bankAccountNumber	Body Param	Numeric	Bank Account Number of the Shop	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
bankIfscCode	Body Param	Alphanumeric	IFSC Code of the company's bank account	Mandatory
bankName	Body Param	Numeric	Bank Name of the registered account. Partner has to pass Bank code from the Bank List API.	Mandatory
shopDistrict	Body Param	Alphabetic	District of the merchant's shop. Merchant will pass code from District List API	Mandatory
lat	Body Param	Numeric	Latitude of the user from where the registration is to be happen	Mandatory
long	Body Param	Numeric	Longitude of the user from where the registration is to be happen	Mandatory
shopPinCode	Body Param	Numeric	Pin Code of the Merchant's shop	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
merchantaddress1	Body Param	Alphanumeric	Address of the merchant's residence	Mandatory
merchantaddress2	Body Param	Alphanumeric	Address of the merchant's residence	Non-Mandatory
merchantState	Body Param	Alphabetic	Merchant's State ID Merchant will pass the code from State List API	Mandatory
ipAddress	Body Param	Numeric	IP Address from where the Merchant Registration will take place.	Mandatory
gender	Body Param	Alphabetic	Gender of the Merchant. Eg. M,F,O.	Mandatory
montherName	Body Param	Alphabetic	Name of the Mother of the user.	Mandatory
fatherName	Body Param	Alphabetic	Name of the Father of the user.	Mandatory
maritalStatus	Body Param	Alphabetic	Marital status of the user. Eg. SINGLE/MARRIED	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
aadhaarNumber	Body Param	Numeric	Aadhaar Number of the Merchant.	Mandatory
merchantPan	Body Param	Alphanumeric	Pan Card Number of the Merchant	Mandatory
shopState	Body Param	Alphabetic	State of the registered shop of the merchant.	Mandatory
merchantdistrict	Body Param	Alphabetic	District of the merchant. Merchant will pass code from District List API	Mandatory
pipe	Body Param	Numeric	Name of the pipe which is being used for AePS transaction	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
merchantRefId	Body Param	Alphanumeric	Merchant will pass unique ID for each request	Mandatory
shopLat	Body Param	Numeric	Latitude of the shop	Mandatory
shopLong	Body Param	Numeric	Longitude of the shop	Mandatory
shopName	Body Param	Alphabetic	Name of the shop	Mandatory
mcc	Body Param	Alphabetic	MCC code of the business	Mandatory
deviceIP	Body Param	Numeric	IP Address from where the Merchant Registration will take place.	Mandatory



• Postman Request to pass Header

Params	Authorization	Headers (10)	Body ●	Scripts	Tests	Settings	Cookies
☒	User-Agent	☐	PostmanRuntime/7.40.0				
☒	Accept	☐	*/*				
☒	Accept-Encoding	☐	gzip, deflate, br				
☒	Connection	☐	keep-alive				
☒	userId		*****				
☒	Authorization		Bearer KL2yN7QEeBGfzF/iuoHZV3xz9Z8YXD...				
	Key		Value				Description

Sample Plain Text Payload

```
{  
  "merchantRefId": "amit1234",  
  "ipAddress": "90.64.212.135",  
  "lat": "28.6273",  
  "long": "77.3725",  
  "firstName": "Amit",  
  "lastName": "Maddheshiya",  
  "middleName": "",  
  "dob": "15-07-1999",  
  "merchantPhoneNumber": "8354045667",  
  "merchantAddress1": "Surajkund Subhash nagar colony",  
  "merchantAddress2": "Gorakhpur Uttar pradesh",  
  "merchantState": "DL",  
  "merchantDistrict": "MAY",  
  "gender": "M",  
  "shopName": "Naren Kirana Store",  
  "motherName": "Katrina Kaif",  
  "fatherName": "Akshay Kumar",  
  "maritalStatus": "MARRIED",  
}
```

```
"mcc": "5045",  
"deviceIP": "150.242.149.135",  
"merchantPinCode": "273001",  
"emailId": "amitmad143@gmail.com",  
"merchantPan": "FOLPM2337L",  
"aadhaarNumber": "4121823418327",  
"shopPan": "AFUPY0248K",  
"bankAccountNumber": "18920145213796",  
"bankIfscCode": "BARB0GEETAP",  
"bankName": "013",  
"accountType": "Savings account",  
"shopAddress": "Surajkund Subhash nagar colony Gorakhpur Uttar pradesh",  
"shopDistrict": "MAY",  
"shopState": "DL",  
"shopPincode": "273001",  
"shopLat": "26.7828",  
"shopLong": "83.3613",  
"pipe": "3"  
}
```

```
<?xml version="1.0" encoding="UTF-8"?><PidOptions ver="1.0">  
<Opts fCount="1" fType="2" format="0" pidVer="2.0" timeout="10000" otp="" posh="UNKNOWN" env="P" wadh="" />  
</PidOptions>
```

Note: If OTP is received from the AePS Txn OTP API, then that OTP will be passed here in the above PID option inside the “otp” parameter.

Note:

When doing Merchant Registration with Face Authentication:

WADH = "mtDVz0PM/HvMAWSkCkjcW+KhNWk2nfbUhfZwLl2faw="

When doing Merchant Registration with Finger:

Wadh = "18f4CEiXeXcfGXvgWA/blxD+w2pw7hfQPY45JMytkPw="

- Sample PID Option For Biometric

```
<?xml version="1.0" encoding="UTF-8"?><PidOptions ver="1.0">  
<Opts fCount="1" fType="2" format="0" pidVer="2.0" timeout="10000" otp="" posh="UNKNOWN" env="P" wadh="" />  
</PidOptions>
```

- Sample PID Option For Face

```
<PidOptions ver="1.0" env="P">  
<Opts fCount="0" fType="0" iCount="0" iType="0" pCount="1" pType="1" format="" pidVer="2.0" timeout="10000" otp="" wadh="..."  
posh="" />  
</PidOptions>
```

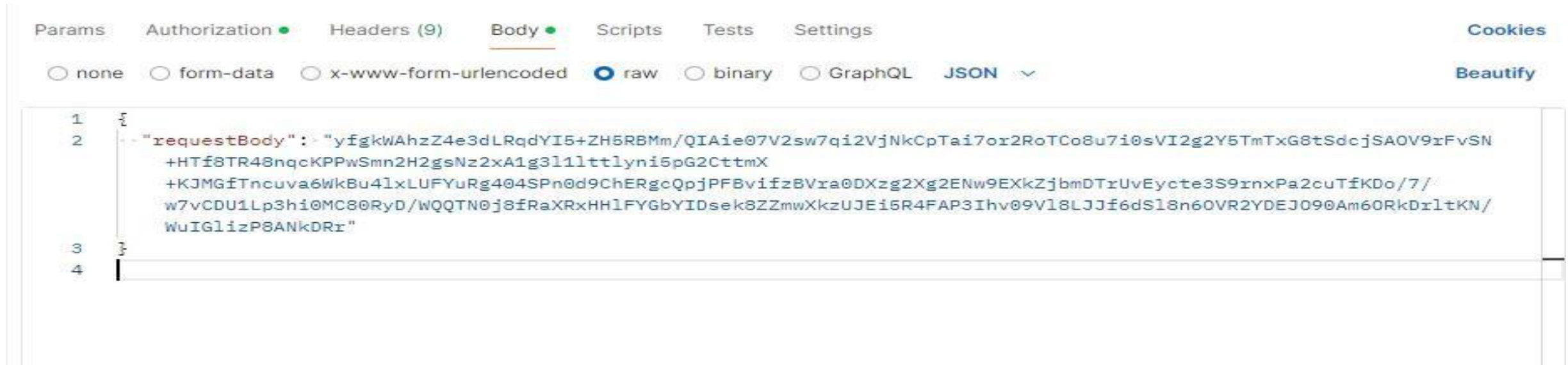
• Sample PID data

```
<PidData>
<Resp errCode="0" errInfo="Success." fCount="1" fType="2" nmPoints="49" qScore="90" />
<DeviceInfo dpld="MANTRA.MSIPL" rdsId="RENESAS.MANTRA.001" rdsVer="1.3.0" mi="MFS110"
mc="fMXPTRhyfwuJJwAa1/dlrV1EK+IHdKH1Lzlc0=" dc="3c43b818-8d35-48cb-a41b-735bae8071b8">
<additional_info>
<Param name="srno" value="8027308" />
<Param name="sysid" value="6C1FEBFACAAE202FBFF0" />
<Param name="ts" value="2025-04-29T17:30:36+05:30" />
<Param name="modality_type" value="Finger" />
<Param name="device_type" value="L1" />
</additional_info>
</DeviceInfo>
<Skey
ci="20250923">em6pZENR5i5YCnT9TLxtgRltAteNrQCCf7c8ig7DdxwzxxCGo+f/wVvqvp2vovDpxBl09Pqr0XtYzzLouZRLj1Mehlky
NuNfQPwsAcd7WKTU7U3Rg==</Skey>
<Hmac>Qj3zT+IlXqdMTO82HTfFpTNSQFdoQzmVqb4/pHLnlyy9tffDs8z9OWaTQ4KXR+AX</Hmac>
<Data type="X">MjAyNS0wNC0yOFkn5jCLbs/CFky+dzBzg==</Data>
</PidData>
```

• Sample Plain Text Payload

```
{  
  "requestBody":  
  "Gll4gsT8Hf+xV8PofIIh6PQ30AdF5ldYZeDPqivup4783W8M5rSH5Juh2y2hF7+Dyv/His4JVU6ZZ6haYTml+Bksucfa4+lKCHnePw1rabf8  
  qHlCfMzP0duODq6sz5EVlwPBLqva+9JrVtcmrAvq/YEcXPTTUs9pisu5BN2BVWy8ElNu4t0YPoyqMRdzVmMh"  
}
```

Postman Request to pass Body Parameter



• Sample Response with Encrypted Data

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data":  
    "1aHQwl94wcBE1ISfbx9RwF0erJS7rDrMx+3B0oHzVj4a5lsbhwcScd8T4D2GodkBti858BhRDSB5v5rKll7KuS6xmOaXTXD0mqUgu/dwQKN  
WYSniD9x9XcSRAVZeoVE0AtMgxXJ8Bm36VwqXoCCbhi7Zocl4Eeib2M99UJm+Np0/+H6Vnt9gzW524391nOTn3laUf46lu5riS3Htmwa1gY  
K1nNvL4jTuAaM5GQTCzgWCyzlDgvrZk7k3fFPsJ9+fzAgocJQ364UcnhxbV+m8vUORC0ojUGJzBOQi9tdDuC+Jgl8ZSZwztk7Gyd9gCZV4ih  
ZsNlatoQVns7G5QkboGe47WN+fiMyxS2kzZorGWgRNU7xhV94BYjObQxO9/mSlrHd/WT1uPolSyXhy6sj3s4GW/lxguZ0nFyihDG8ZNRKna  
oY1Wz91d5Tc7fyeWpJiRI9ZohoEgs0rQk/r6zzRty2IEwe6EokKHfdYGOnccqeVldQONCjRvhCJriJjpiEhsGPAdBB43Zil5/XJA7pPeber1S5P  
G9cOcGG/jWyQReUGg639Z/i4mccpAqRZP9vekOjnaceapzB84e/6VqX929le2lxCzJiHLrIwWLBPHo4h5xSFlq9/Q+ARzzLiyCdNIGqt+ihlWi  
UUWbj0HQJaUKSt/28aV6fxIsOBFluQDXewfljUCy6BJjtT964s05WCWdj4RmcJWXJiAhTX7rJbWEDUIDCJc3Shrli9OKC1Qj/J4W0Mk7Ywc  
ptll62rp74479odNVFfKUUNHpBxbHa7dfBUEP6J9KzTjP1zZ2pNZyVMWPgatVSLVrFlRDMmDd5PIW0nxzunjlBYSDVKq7oTI64Eii4K9dPb3v  
L/o581qpYKCKQKD8ina3RIs3mS+QSBh4Mk9ln708P6RbM1KjoFmO6o8hYySjQPb+BsbyhpjHg0Dt6Sdcz1fhYdG"  
}
```

Sample Response with Decrypted Data

```
{  
  "status": "000",  
  "merchantStatus": "Success",  
  "statusDescription": "Registration Done, OTP verification pending! ",  
  "merchantId": "MER1234567",  
  "txnRefId": "3f31b43f0f1c4c38a933d2b613183072",  
  "merchantRefId": "amit1234",  
  "firstName": "Amit",  
  "middleName": "",  
  "lastName": "Maddheshiya",  
  "emailId": "amitmad143@gmail.com",  
  "mobileNo": "8354045667",  
  "aadhaarNo": "XXXXXXXXX8327",  
  "panNo": "FOLPM2337L",  
  "pipe": "1"  
}
```

Send OTP

- **URL** - /aepsapi/api/payment/sendotpuat
- **Method Type** - HTTP POST
- **Description** - This API is used to send OTP for mobile number verification.
- **Authorization Type** - Bearer Token (token value which we received from the partner authorization API call need to pass here.)

Headers Parameter	Type	Data Type	Description	Requisite
Bearer Token	Authorization	Alphanumeric	token value which you received from the partner authorization API.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
merchantId	Body Param	Alphanumeric	Partner will pass the merchant ID which they receive while doing merchant registration	Mandatory
pipe	Body Param	Numeric	Name of the pipe which is being used for AePS transaction	Mandatory
merchantRefId	Body Param	Alphanumeric	Merchant will pass unique ID for each request	Mandatory

- Postman Request to pass Header

Params	Authorization	Headers (10)	Body	Scripts	Tests	Settings	Cookies
☒	User-Agent	☒	PostmanRuntime/7.40.0				
☒	Accept	☒	*/*				
☒	Accept-Encoding	☒	gzip, deflate, br				
☒	Connection	☒	keep-alive				
☒	userId		*****				
☒	Authorization		Bearer KL2yN7QEeBGfzF/iuoHZV3xz9Z8YXD...				
	Key		Value				Description

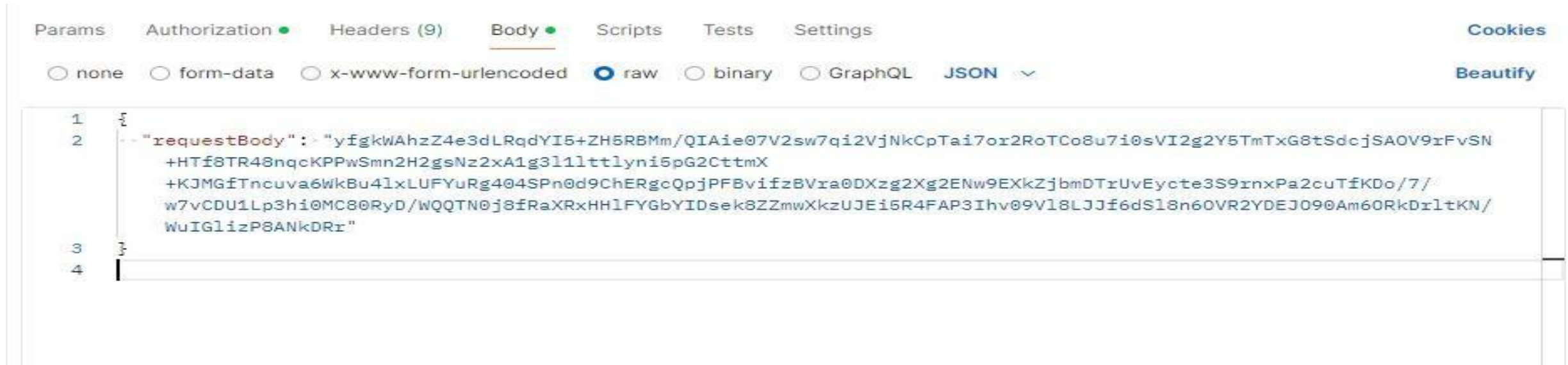
- **Sample Plain Text Payload**

```
{  
  "merchantId": "MER3761446",  
  "merchantRefId": "amit12345",  
  "pipe": "1"  
}
```

• Sample Plain Text Payload

```
{  
  "requestBody":  
  "Gll4gsT8Hf+xV8PofIIh6PQ30AdF5ldYZeDPqivup4783W8M5rSH5Juh2y2hF7+Dyv/His4JVU6ZZ6haYTml+Bksucfa4+lKCHnePw1rabf8  
  qHlCfMzP0duODq6sz5EVlwPBLqva+9JrVtcmrAvq/YEcXPTTUs9pisu5BN2BVWy8ElNu4t0YPoyqMRdzVmMh"  
}
```

Postman Request to pass Body Parameter



- Sample Response with Encrypted Data

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data": "yfgjThx0TJWuM6xyLL9/8ZX2bRgK+A8ewv0nZmsmkP6xDjl/aMuA2+EW1ilvHala6D1NWJeB6  
bAlqh1N2+GGbBqUAgd8leDF+xi/wARe8myJEZTQUj3+GQBU CXCKQXNunG1RsBKjndpJyhNAw  
ELOaeyJPmEnvrrK6zCpgEGff+7TrNYbS+Cia4eLEhYAab2/PTPnWGhCvNp/YU0GX0yACWR5XW  
hDaxeRuS/j2Zga/yJGij+h88yDeK6b3ZzB1LzxPgj60DPoLXVpQTP4alc1WA2WAFySRTngmW0Fbt  
JBhIF/IK2MG2r/AG+4I/sbeSI98WBkq9XjAvGha0hRocZA6PbHN/xRJDWXdVtLDPdyRt8s1Ifua0hQKJBF+cxe1MWV/OqwUEzcggen7uYE  
+2hXsEa4QaLfE+zWRyFEJJGqslbxmVbeFAnx6EGkbbAr0/VEQUC3/oi19ZwZ7IB"  
}
```

- Sample Response with Decrypted Data

```
{  
  "status": "000",  
  "merchantStatus": "Success",  
  "statusDescription": "OTP Send",  
  "merchantRefId": "amit12345",  
  "merchantId": "MER3761446",  
  "txnRefId": "c26da1d7c1b44624909b6cf1da30ec70",  
  "pipe": "1"  
}
```

Resend OTP

- **URL** - /aepsapi/api/payment/resendotpuat
- **Method Type** - HTTP POST
- **Description** - This API is used to resend OTP for mobile number verification.
- **Authorization Type** - Bearer Token (token value which we received from the partner authorization API call need to pass here.)

Headers Parameter	Type	Data Type	Description	Requisite
Bearer Token	Authorization	Alphanumeric	token value which you received from the partner authorization API.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
merchantId	Body Param	Alphanumeric	Partner will pass the merchant ID which they receive while doing merchant registration	Mandatory
pipe	Body Param	Numeric	Name of the pipe which is being used for AePS transaction	Mandatory
merchantRefId	Body Param	Alphanumeric	Merchant will pass unique ID for each request	Mandatory

- Postman Request to pass Header

Params	Authorization	Headers (10)	Body	Scripts	Tests	Settings	Cookies
☒	User-Agent	☒	PostmanRuntime/7.40.0				
☒	Accept	☒	*/*				
☒	Accept-Encoding	☒	gzip, deflate, br				
☒	Connection	☒	keep-alive				
☒	userId		*****				
☒	Authorization		Bearer KL2yN7QEeBGfzF/iuoHZV3xz9Z8YXD...				
	Key		Value				Description

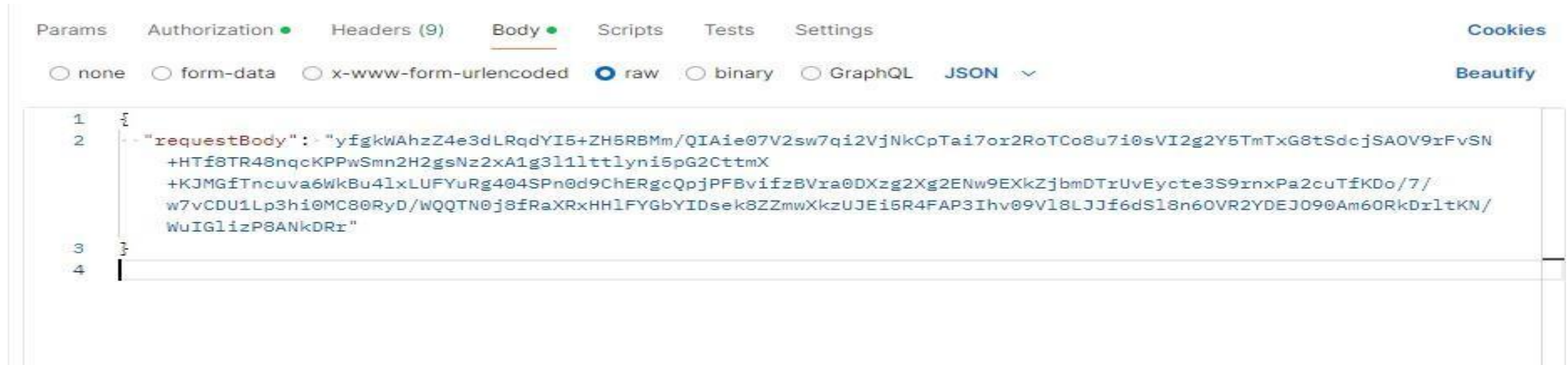
- Sample Plain Text Payload

```
{  
  "merchantId": "MER3761446",  
  "merchantRefId": "amit12345",  
  "pipe": "1"  
}
```

• Sample Plain Text Payload

```
{  
  "requestBody":  
  "Gll4gsT8Hf+xV8PofIIh6PQ30AdF5ldYZeDPqivup4783W8M5rSH5Juh2y2hF7+Dyv/His4JVU6ZZ6haYTml+Bksucfa4+lKCHnePw1rabf8  
  qHlCfMzP0duODq6sz5EVlwPBLqva+9JrVtcmrAvq/YEcXPTTUs9pisu5BN2BVWy8ElNu4t0YPoyqMRdzVmMh"  
}
```

Postman Request to pass Body Parameter



• Sample Response with Encrypted Data

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data": "yfgjThx0TJWuM6xyLL9/8ZX2bRgK+A8ewv0nZmsmkP6xDjI/aMuA2+EW1ilvHala6D1NWJeB6  
bAlqh1N2+GGbBqUAgd8leDF+xi/wARe8myJEZTQUj3+GQBU CXCKQXNunG1RsBKjndpJyhNAw  
ELOaeyJPmEnvrrK6zCpgEGff+7TrNYbS+Cia4eLEhYAab2/PTPnWGhCvNp/YU0GX0yACWR5XW  
hDaxeRuS/j2Zga/yJGij+h88yDeK6b3ZzB1LzxPgj60DPoLXVpQTP4alc1WA2WAFySRTngmW0Fbt  
JBhIF/IK2MG2r/AG+4I/sbeSI98WBkq9XjAvGha0hRocZA6PbHN/xRJDW XDVTLDpdyRt8s1Ifua0hQKJBF+cxe1MWV/OqwUEzcggen7uYE  
+2hXsEa4QaLfE+zWRyFEJJGqslbxmVbeFAnx6EGkbbAr0/VEQUC3/oi19ZwZ7IB"  
}
```

- Sample Response with Decrypted Data

```
{  
  "status": "000",  
  "merchantStatus": "Success",  
  "statusDescription": "OTP Send",  
  "merchantRefId": "amit12345",  
  "merchantId": "MER3761446",  
  "txnRefId": "c26da1d7c1b44624909b6cf1da30ec70",  
  "pipe": "1"  
}
```

Validate OTP

- **URL** - /aepsapi/api/payment/validateotpuat
- **Method Type** - HTTP POST
- **Description** - This API is used to validate OTP for mobile number verification.
- **Authorization Type** - Bearer Token (token value which we received from the partner authorization API call need to pass here.)

Headers Parameter	Type	Data Type	Description	Requisite
Bearer Token	Authorization	Alphanumeric	token value which you received from the partner authorization API.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
merchantId	Body Param	Alphanumeric	Partner will pass the merchant ID which they receive while doing merchant registration	Mandatory
pipe	Body Param	Numeric	Name of the pipe which is being used for AePS transaction	Mandatory
merchantRefId	Body Param	Alphanumeric	Merchant will pass unique ID for each request	Mandatory
otp	Body Param	Numeric	OTP received on mobile number which was used in merchant registration	Mandatory

- Postman Request to pass Header

Params	Authorization	Headers (10)	Body	Scripts	Tests	Settings	Cookies
☒	User-Agent	☒	PostmanRuntime/7.40.0				
☒	Accept	☒	*/*				
☒	Accept-Encoding	☒	gzip, deflate, br				
☒	Connection	☒	keep-alive				
☒	userId		*****				
☒	Authorization		Bearer KL2yN7QEeBGfzF/iuoHZV3xz9Z8YXD...				
	Key		Value				Description

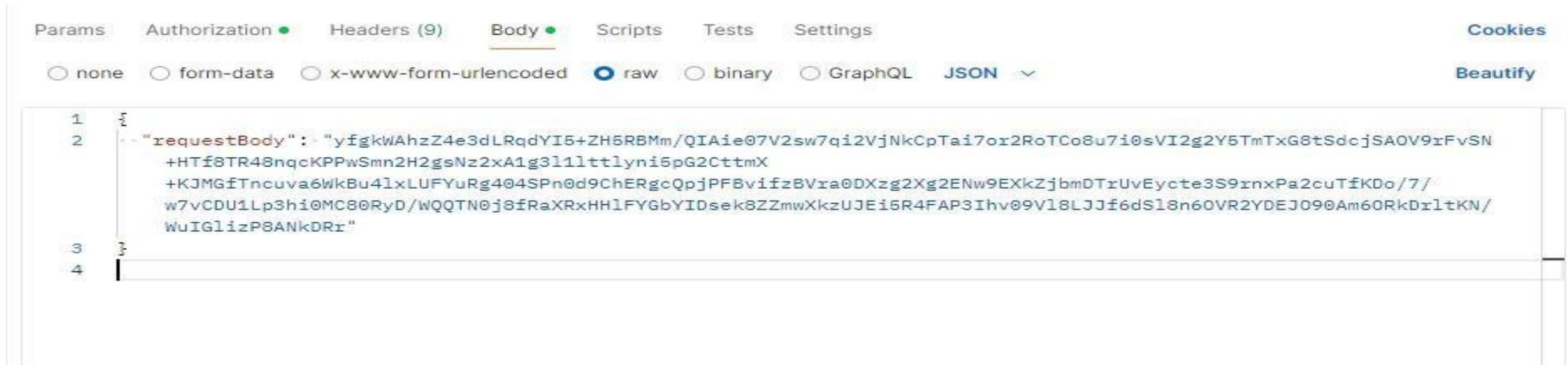
- **Sample Plain Text Payload**

```
{  
  "merchantRefId": "amit12345",  
  "merchantId": "MER3761446",  
  "otp": "155872",  
  "pipe": "1"  
}
```

• Sample Plain Text Payload

```
{  
  "requestBody":  
  "Gll4gsT8Hf+xV8PofIIh6PQ30AdF5IdYZeDPqivup4783W8M5rSH5Juh2y2hF7+Dyv/His4JVU6ZZ6haYTml+Bksucfa4+lKCHnePw1rabf8  
  qHlCfMzP0duODq6sz5EVlwPBLqva+9JrVtcmrAvq/YEcXPTTUs9pisu5BN2BVWy8ElNu4t0YPoyqMRdzVmMh"  
}
```

Postman Request to pass Body Parameter



- Sample Response with Encrypted Data

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data": "yfgjThx0TJWuM6xyLL9/8ZX2bRgK+A8ewv0nZmsmkP6xDjl/aMuA2+EW1ilvHala6D1NWJeB6  
bAlqh1N2+GGbBqUAgd8leDF+xi/wARe8myJEZTQUj3+GQBUcXCKQXNunG1RsBKjndpJyhNAw  
ELOaeyJPmEnvrrK6zCpgEGff+7TrNYbS+Cia4eLEhYAab2/PTPnWGhCvNp/YU0GX0yACWR5XW  
hDaxeRuS/j2Zga/yJGij+h88yDeK6b3ZzB1LzxPgj60DPoLXVpQTP4alc1WA2WAFySRTngmW0Fbt  
JBhIF/IK2MG2r/AG+4I/sbeSI98WBkq9XjAvGha0hRocZA6PbHN/xRJDWXdVtLDPdyRt8s1Ifua0hQKJBF+cxe1MWV/OqwUEzcg7uYE  
+2hXsEa4QaLfE+zWRyFEJJGqslbxmVbeFAnx6EGkbbAr0/VEQUC3/oi19ZwZ7IB"  
}
```

- Sample Response with Decrypted Data

```
{  
  "status": "000",  
  "merchantStatus": "Success",  
  "statusDescription": "OTP VERIFIED.",  
  "merchantRefId": "amit12345",  
  "merchantId": "MER3761446",  
  "txnRefId": "2ae85886134949258cc67e6ffa1975a2",  
  "pipe": "1"  
}
```

AePS Txn OTP

- **URL** - /aepsapi/api/Payment/AepsTransactionOtpuat
- **Method Type** - HTTP POST
- **Description** - This API is used to send OTP for CW and AP transactions exceeding Rs. 5000.
- **Authorization Type** - Bearer Token (token value which we received from the partner authorization API call need to pass here.)

Headers Parameter	Type	Data Type	Description	Requisite
BearerToken	Authorization	Alphanumeric	token value which you received from the partner authorization API.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
merchantId	Body Param	Alphanumeric	Partner will pass the merchant ID which they receive while doing merchant registration	Mandatory
pipe	Body Param	Numeric	Name of the pipe which is being used for AePS transaction	Mandatory
merchantRefId	Body Param	Alphanumeric	Merchant will pass unique ID for each request	Mandatory
aadhaarNumber	Body Param	Numeric	Aadhaar Number of the Merchant.	Mandatory
bankIIN	Body Param	Numeric	Merchant will pass Bank IIN number from the given API.	Mandatory
transactionType	Body Param	Alphabetic	User needs to define type of Transaction ie. CWTFEA/APTFEA	Mandatory
amount	Body Param	Numeric	Transaction Amount	Mandatory
mobileNumber	Body Param	Numeric	Mobile Number of the merchant.	Mandatory
custMobileNumber	Body Param	Numeric	Mobile Number of the customer.	Non-Mandatory

Body Parameter	Type	Data Type	Description	Requisite
lat	Body Param	Numeric	Latitude of the user from where the transaction is to be happen	Mandatory
long	Body Param	Numeric	Longitude of the user from where the transaction is to be happen	Mandatory
ipAddress	Body Param	Numeric	IP Address from where the AePs transaction will take place.	Mandatory
appPlatform	Body Param	Numeric	Platform of the partner application making the request ie. Android, Web, SDK, etc	Mandatory
appVersion	Body Param	Numeric	Version of the partner application making the request	Mandatory

- Postman Request to pass Header

Params	Authorization	Headers (10)	Body	Scripts	Tests	Settings	Cookies
☒	User-Agent	☒	PostmanRuntime/7.40.0				
☒	Accept	☒	*/*				
☒	Accept-Encoding	☒	gzip, deflate, br				
☒	Connection	☒	keep-alive				
☒	userId		*****				
☒	Authorization		Bearer KL2yN7QEeBGfzF/iuoHZV3xz9Z8YXD...				
	Key		Value				Description

Sample Plain Text Payload

Note: If pipe is 4 and bank is Airtel Payments Bank, then this OTP is not required and the user can directly proceed with the transaction.

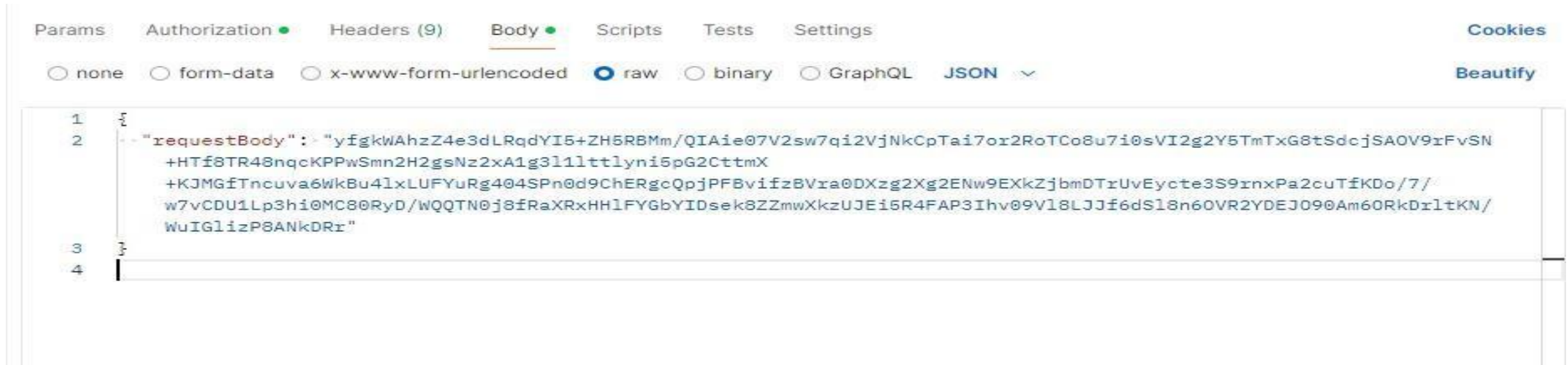
Note: Customer receives OTP on the Aadhaar-registered mobile number.

```
{  
  "merchantRefId": "901B8C3BEF954110A1AF60934E1C9180",  
  "merchantId": "MER2242972",  
  "bankIIN": "607094",  
  "aadhaarNumber": "279268483126",  
  "transactionType": "APTFA",  
  "amount": "5100.0000",  
  "mobileNumber": "8285343230",  
  "custMobileNumber": "",  
  "lat": "28.6271",  
  "long": "77.3723",  
  "ipAddress": "203.122.15.186",  
  "pipe": "2",  
  "appPlatform": "android",  
  "appVersion": "1.0.0"  
}
```

• Sample Plain Text Payload

```
{  
  "requestBody":  
  "Gll4gsT8Hf+xV8PofIIh6PQ30AdF5ldYZeDPqivup4783W8M5rSH5Juh2y2hF7+Dyv/His4JVU6ZZ6haYTml+Bksucfa4+lKCHnePw1rabf8  
  qHlCfMzP0duODq6sz5EVlwPBLqva+9JrVtcmrAvq/YEcXPTTUs9pisu5BN2BVWy8ElNu4t0YPoyqMRdzVmMh"  
}
```

Postman Request to pass Body Parameter



• Sample Response with Encrypted Data

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data": "yfgjThx0TJWuM6xyLL9/8ZX2bRgK+A8ewv0nZmsmkP6xDjI/aMuA2+EW1ilvHala6D1NWJeB6  
bAlqh1N2+GGbBqUAgd8leDF+xi/wARe8myJEZTQUj3+GQBU CXCKQXNunG1RsBKjndpJyhNAw  
ELOaeyJPmEnvrrK6zCpgEGff+7TrNYbS+Cia4eLEhYAab2/PTPnWGhCvNp/YU0GX0yACWR5XW  
hDaxeRuS/j2Zga/yJGij+h88yDeK6b3ZzB1LzxPgj60DPoLXVpQTP4alc1WA2WAFySRTngmW0Fbt  
JBhIF/IK2MG2r/AG+4I/sbeSI98WBkq9XjAvGha0hRocZA6PbHN/xRJDWXdVtLDPdyRt8s1Ifua0hQKJBF+cxe1MWV/OqwUEzcggen7uYE  
+2hXsEa4QaLfE+zWRyFEJJGqslbxmVbeFAnx6EGkbbAr0/VEQUC3/oi19ZwZ7IB"  
}
```

- **Sample Response with Decrypted Data**

```
{  
  "npciCode": "00",  
  "npciMessage": "OTP Send Successfully",  
  "status": "000",  
  "statusDescription": "Transaction Success.",  
  "txnDateTime": "05-08-2026 12:05:01",  
  "txnRefId": "FNU62171200616099999920260805120456,006160,621712006160"  
}
```

- **URL** - /aepsapi/api/payment/merchantekycuat
- **Method Type** - HTTP POST
- **Description** - This API is used to validate E-KYC.
- **Authorization Type** - Bearer Token (token value which we received from the partner authorization API call need to pass here.)

Headers Parameter	Type	Data Type	Description	Requisite
Bearer Token	Authorization	Alphanumeric	token value which you received from the partner authorization API.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
merchantId	Body Param	Alphanumeric	Partner will pass the merchant ID which they receive while doing merchant registration	Mandatory
pipe	Body Param	Numeric	Name of the pipe which is being used for AePS transaction	Mandatory
pidData	Body Param	Alphanumeric	Merchant will pass merchant's Biometric data.	Mandatory
merchantRefId	Body Param	Alphanumeric	Merchant will pass unique ID for each request	Mandatory

- Postman Request to pass Header

Params	Authorization	Headers (10)	Body	Scripts	Tests	Settings	Cookies
☒	User-Agent	☒	PostmanRuntime/7.40.0				
☒	Accept	☒	*/*				
☒	Accept-Encoding	☒	gzip, deflate, br				
☒	Connection	☒	keep-alive				
☒	userId		*****				
☒	Authorization		Bearer KL2yN7QEeBGfzF/iuoHZV3xz9Z8YXD...				
	Key		Value				Description

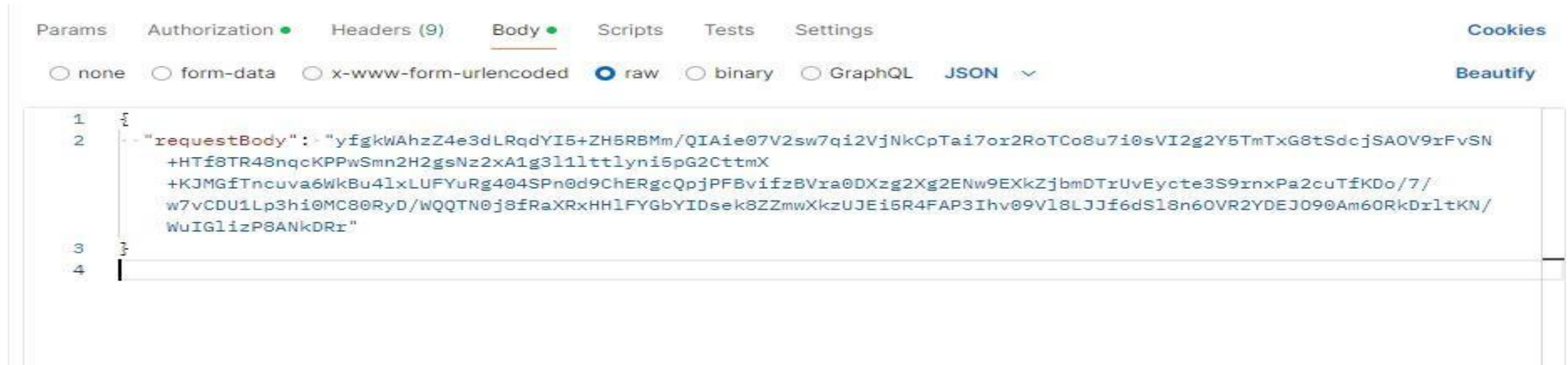
- Sample Plain Text Payload

```
{  
  "merchantRefId": "amit345",  
  "merchantId": "MER6453620",  
  "pipe": "1",  
  "pidData": "<?xml version=\"1.0\"?>\r\n<PidData>\r\n  <Resp errCode=\"0\" errInfo=\"Success.\" fCount=\"1\"  
fType=\"2\" nmPoints=\"51\" qScore=\"60\" />\r\n  <DeviceInfo dpId=\"MANTRA.MSIPL\"  
rdsId=\"RENESAS.MANTRA.001\" rdsVer=\"1.3.0\" mi=\"MFS110\"  
mc=\"MIIEADCCAuigAwIBAgIIQkNENEU3QkMwDQYJKoZIhvcNAQELBQAwwgfwxKjAo  
}
```

• Sample Plain Text Payload

```
{  
  "requestBody":  
  "Gll4gsT8Hf+xV8PofIIh6PQ30AdF5ldYZeDPqivup4783W8M5rSH5Juh2y2hF7+Dyv/His4JVU6ZZ6haYTml+Bksucfa4+lKCHnePw1rabf8  
  qHlCfMzP0duODq6sz5EVlwPBLqva+9JrVtcmrAvq/YEcXPTTUs9pisu5BN2BVWy8ElNu4t0YPoyqMRdzVmMh"  
}
```

Postman Request to pass Body Parameter



• Sample Response with Encrypted Data

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data": "yfgjThx0TJWuM6xyLL9/8ZX2bRgK+A8ewv0nZmsmkP6xDjl/aMuA2+EW1ilvHala6D1NWJeB6  
bAlqh1N2+GGbBqUAgd8leDF+xi/wARe8myJEZTQUj3+GQBU CXCKQXNunG1RsBKjndpJyhNAw  
ELOaeyJPmEnvrrK6zCpgEGff+7TrNYbS+Cia4eLEhYAab2/PTPnWGhCvNp/YU0GX0yACWR5XW  
hDaxeRuS/j2Zga/yJGij+h88yDeK6b3ZzB1LzxPgj60DPoLXVpQTP4alc1WA2WAFySRTngmW0Fbt  
JBhIF/IK2MG2r/AG+4I/sbeSI98WBkq9XjAvGha0hRocZA6PbHN/xRJDWXdVtLDPdyRt8s1Ifua0hQKJBF+cxe1MWV/OqwUEzcggen7uYE  
+2hXsEa4QaLfE+zWRyFEJJGqslbxmVbeFAnx6EGkbbAr0/VEQUC3/oi19ZwZ7IB"  
}
```

- Sample Response with Decrypted Data

```
{  
  "status": "000",  
  "merchantStatus": "Success",  
  "statusDescription": "KYC Done!",  
  "merchantRefId": "amit345",  
  "merchantId": "MER6453620",  
  "txnRefId": "8d902ce7c8384b9ab5b7d7712ba0dd6c",  
  "pipe": "1"  
}
```

2 Factor Authentication

- **URL** - /aepsapi/api/payment/2fauat
- **Method Type** - HTTP POST
- **Description** - This API is used to 2 factor authentication. For each request merchant has to pass a unique merchant ref Id. Merchant must be active to initiate any financial or non-financial transaction. Merchant has to do 2-FA once in a day.
- **Authorization Type** - Bearer Token (token value which we received from the partner authorization API call need to pass here.)

Headers Parameter	Type	Data Type	Description	Requisite
Bearer Token	Authorization	Alphanumeric	token value which you received from the partner authorization API.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
merchantId	Body Param	Alphanumeric	Partner will pass the merchant ID which they receive while doing merchant registration	Mandatory
pipe	Body Param	Numeric	Name of the pipe which is being used for AePS transaction	Mandatory
pidData	Body Param	Alphanumeric	Merchant will pass merchant's Biometric data.	Mandatory
merchantRefId	Body Param	Alphanumeric	Merchant will pass unique ID for each request	Mandatory
aadhaarNumber	Body Param	Numeric	Aadhaar Number of the Merchant.	Mandatory
deviceType	Body Param	Alphabetic	Name of authentication device	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
lat	Body Param	Numeric	Latitude of the user from where the 2FA is to be happen	Mandatory
long	Body Param	Numeric	Longitude of the user from where the 2FA is to be happen	Mandatory

• Postman Request to pass Header

Params	Authorization	Headers (10)	Body ●	Scripts	Tests	Settings	Cookies
☒	User-Agent	☐	PostmanRuntime/7.40.0				
☒	Accept	☐	*/*				
☒	Accept-Encoding	☐	gzip, deflate, br				
☒	Connection	☐	keep-alive				
☒	userId		*****				
☒	Authorization		Bearer KL2yN7QEeBGfzF/iuoHZV3xz9Z8YXD...				
	Key		Value				Description

- **Sample Plain Text Payload (For Biometric)**

```
{  
  "merchantRefId": "amit345",  
  "merchantId": "MER6453620",  
  "aadhaarNumber": "4121823418327",  
  "pipe": "1",  
  "deviceType": "mantra",  
  "pidData": "<?xml version=\"1.0\"?>\r\n<PidData>\r\n  <Resp errCode=\"0\" errInfo=\"Success.\" fCount=\"1\"  
fType=\"2\" nmPoints=\"51\" qScore=\"60\" />\r\n  <DeviceInfo dpId=\"MANTRA.MSIPL\"  
rdsId=\"RENESAS.MANTRA.001\" rdsVer=\"1.3.0\" mi=\"MFS110\"  
mc=\"MIIEADCCAuigAwIBAgIIQkNENEU3QkMwDQYJKoZIhvcNAQELBQAwwgfwxKjAo\",  
  "lat": "28.7041",  
  "long": "77.1025"  
}
```

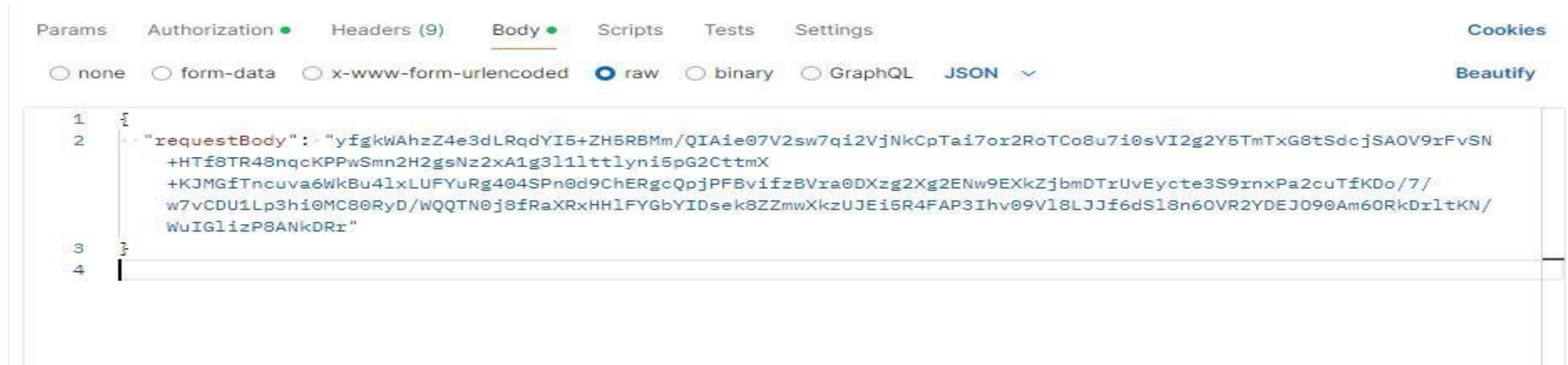
- Sample Plain Text Payload (For Face)

```
{  
  "merchantRefId": "amit345",  
  "merchantId": "MER6453620",  
  "aadhaarNumber": "4121823418327",  
  "pipe": "1",  
  "deviceType": "aadhaarfacerd",  
  "pidData": "<?xml version=\"1.0\"?>\r\n<PidData>\r\n  <Resp errCode=\"0\" errInfo=\"Success.\" fCount=\"1\"  
fType=\"2\" nmPoints=\"51\" qScore=\"60\" />\r\n  <DeviceInfo dpId=\"MANTRA.MSIPL\"  
rdsId=\"RENESAS.MANTRA.001\" rdsVer=\"1.3.0\" mi=\"MFS110\"  
mc=\"MIIEADCCAuigAwIBAgIIQkNENEU3QkMwDQYJKoZIhvcNAQELBQAwwgfwxKjAo“,  
  "lat": "28.7041",  
  "long": "77.1025"  
}
```

• Sample Plain Text Payload

```
{  
  "requestBody":  
  "Gll4gsT8Hf+xV8PofIIh6PQ30AdF5ldYZeDPqivup4783W8M5rSH5Juh2y2hF7+Dyv/His4JVU6ZZ6haYTml+Bksucfa4+lKCHnePw1rabf8  
  qHlCfMzP0duODq6sz5EVlwPBLqva+9JrVtcmrAvq/YEcXPTTUs9pisu5BN2BVWy8ElNu4t0YPoyqMRdzVmMh"  
}
```

Postman Request to pass Body Parameter



• Sample Response with Encrypted Data

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data":  
    "1aHQwl94wcBE1ISfbx9RwF0erJS7rDrMx+3B0oHzVj4a5lsbhwcScd8T4D2GodkBti858BhRDSB5v5rKll7KuS6xmOaXTXD0mqUgu/dwQKN  
WYSniD9x9XcSRAVZeoVE0AtMgxXJ8Bm36VwqXoCCbhi7Zocl4Eeib2M99UJm+Np0/+H6Vnt9gzW524391nOTn3laUf46lu5riS3Htmwa1gY  
K1nNvL4jTuAaM5GQTCzgWCyzlDgvrZk7k3fFPsJ9+fzAgocJQ364UcnhxbV+m8vUORC0ojUGJzBOQi9tdDuC+Jgl8ZSZwztk7Gyd9gCZV4ih  
ZsNlatoQVns7G5QkboGe47WN+fiMyxS2kzZorGWgRNU7xhV94BYjObQxO9/mSlrHd/WT1uPolSyXhy6sj3s4GW/lxguZ0nFyihDG8ZNRKna  
oY1Wz91d5Tc7fyeWpJiRI9ZohoEgs0rQk/r6zzRty2IEwe6EokKHfdYGOnccqeVldQONCjRvhCJriJjpiEhsGPAdBB43Zil5/XJA7pPeber1S5P  
G9cOcGG/jWyQReUGg639Z/i4mccpAqRZP9vekOjnaceapzB84e/6VqX929le2IxCzJiHLrIwWLBPHo4h5xSFlq9/Q+ARzzLiyCdNIGqt+ihlWi  
UUWbj0HQJaUKSt/28aV6fxIsOBFluQDXewfljUCy6BJjtT964s05WCWdj4RmcJWXJiAhTX7rJbWEDUIDCJc3Shrli9OKC1Qj/J4W0Mk7Ywc  
ptll62rp74479odNVFfKUUNHpBxbHa7dfBUEP6J9KzTjP1zZ2pNZyVMWPgatVSLVrFlRDMmDd5PIW0nxzunjlBYSDVKq7oTI64Eii4K9dPb3v  
L/o581qpYKCKQKD8ina3RIs3mS+QSBh4Mk9ln708P6RbM1KjoFmO6o8hYySjQPb+BsbyhpjHg0Dt6Sdcz1fhYdG"  
}
```

Sample Response with Decrypted Data

```
{  
  "status": "000",  
  "merchantStatus": "Success",  
  "statusDescription": "Two FA Done!",  
  "merchantRefId": "amit765",  
  "merchantId": "MER6453620",  
  "txnRefId": "49f000e3e2ad4571a2fa1b53c1a93a3c",  
  "pipe": "1",  
  "lat": "28.7041",  
  "long": "77.1025"  
}
```

AePS Transaction

- **URL** - /aepsapi/api/payment/aepsuat
- **Method Type** - HTTP POST
- **Description** - This API is used to initiate an AePS Transaction. For each request merchant has to pass a unique merchant ref Id. Merchant must be active to initiate any financial or non-financial transaction.
- **Authorization Type** - Bearer Token (token value which we received from the partner authorization API call need to pass here.)

Headers Parameter	Type	Data Type	Description	Requisite
Bearer Token	Authorization	Alphanumeric	token value which you received from the partner authorization API.	Mandatory
userId	Authorization	Alphanumeric	User Id, partner will get this key at the time of partner's onboarding. This key is used for authorization purpose.	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
transactionType	Body Param	Alphabetic	User needs to define type of Transaction. Eg. - Cash Withdrawl(CW) - Cash Deposit (CD), - Balance Enquiry(BE), etc	Mandatory
amount	Body Param	Numeric	Amount to be transfer. Amount can't be less than 100 rupees. Range will be in between INR 100-10000. Note: In case of balance enquiry and mini-statement transaction Amount will 0.	Mandatory
aadhaarNumber	Body Param	Numeric	Aadhaar Number of the merchant.	Mandatory
bankIIN	Body Param	Numeric	Merchant will pass Bank IIN number from the given API.	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
merchantId	Body Param	Numeric	Merchant will pass their merchant ID.	Mandatory
mobileNo	Body Param	Alphanumeric	Mobile Number of the merchant.	Mandatory
ipAddress	Body Param	Numeric	IP Address from where the AePs transaction will take place.	Mandatory
pidData	Body Param	Alphanumeric	Merchant will pass customer's Biometric data.	Mandatory
pipe	Body Param	Numeric	Name of the pipe which is being used for AePS transaction	Mandatory
merchantRefId	Body Param	Alphanumeric	Merchant will pass unique ID for each request	Mandatory
deviceType	Body Param	Alphabetic	Name of authentication device	Mandatory

Body Parameter	Type	Data Type	Description	Requisite
lat	Body Param	Numeric	Latitude of the user from where the registration is to be happen	Mandatory
long	Body Param	Numeric	Longitude of the user from where the registration is to be happen	Mandatory
udf1	Body Param	Alphanumeric	Merchant can pass any value for reference	Non- Mandatory
udf2	Body Param	Alphanumeric	Merchant can pass any value for reference	Non- Mandatory
udf3	Body Param	Alphanumeric	Merchant can pass any value for reference	Non- Mandatory
cwAuthTxnId	Body Param	Alphanumeric	txnRef ID from the AePS Txn OTP API	Mandatory

• Postman Request to pass Header

Params	Authorization	Headers (10)	Body ●	Scripts	Tests	Settings	Cookies
☒	User-Agent		PostmanRuntime/7.40.0				
☒	Accept		*/*				
☒	Accept-Encoding		gzip, deflate, br				
☒	Connection		keep-alive				
☒	userId		*****				
☒	Authorization		Bearer KL2yN7QEeBGfzF/iuoHZV3xz9Z8YXD...				
	Key		Value				Description

- **Sample Plain Text Payload (For Biometric)**

```
{  
  "merchantRefId": "amit999",  
  "merchantId": "MER6453620",  
  "transactionType": "CW",  
  "aadhaarNumber": "301292940738",  
  "mobileNumber": "8354005667",  
  "amount": "150",  
  "bankIIN": "608117",  
  "ipAddress": "192.168.1.1",  
  "pipe": "1",  
  "lat": "28.7041",  
  "long": "77.1025",  
  "deviceType": "mantra",  
  "cwAuthTxnId": "FNU62171200616099999920260805120456,006160,621712006160",  
}
```

- Sample Plain Text Payload

```
"udf1": "",  
"udf2": "",  
"udf3": ""  
"pidData": "<?xml version=\"1.0\"?>\r\n<PidData>\r\n  <Resp errCode=\"0\" errInfo=\"Success.\" fCount=\"1\"  
fType=\"2\" nmPoints=\"43\" qScore=\"70\" />\r\n  <DeviceInfo dpId=\"MANTRA.MSIPL\"  
rdsId=\"RENESAS.MANTRA  
}
```

- **Sample Plain Text Payload (For Face)**

```
{  
  "merchantRefId": "amit999",  
  "merchantId": "MER6453620",  
  "transactionType": "CW",  
  "aadhaarNumber": "301292940738",  
  "mobileNumber": "8354005667",  
  "amount": "150",  
  "bankIIN": "608117",  
  "ipAddress": "192.168.1.1",  
  "pipe": "1",  
  "lat": "28.7041",  
  "long": "77.1025",  
  "deviceType": "aadhaarfacerd",  
}
```

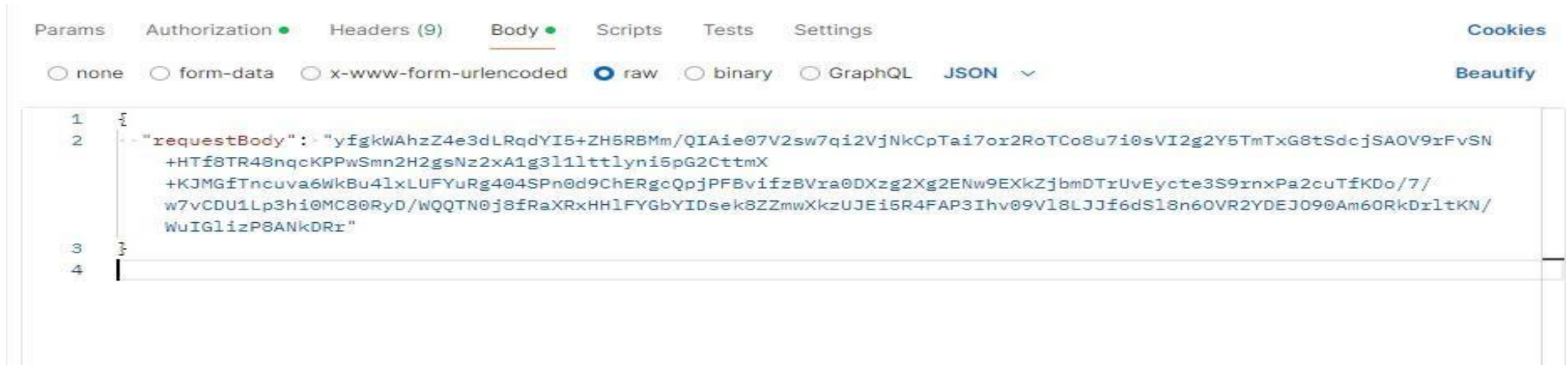
- Sample Plain Text Payload

```
"udf1": "",  
"udf2": "",  
"udf3": ""  
"pidData": "<?xml version=\"1.0\"?>\r\n<PidData>\r\n  <Resp errCode=\"0\" errInfo=\"Success.\" fCount=\"1\"  
fType=\"2\" nmPoints=\"43\" qScore=\"70\" />\r\n  <DeviceInfo dpld=\"MANTRA.MSIPL\"  
rdsld=\"RENESAS.MANTRA  
}
```

• Sample Plain Text Payload

```
{  
  "requestBody":  
  "Gll4gsT8Hf+xV8PofIIh6PQ30AdF5ldYZeDPqivup4783W8M5rSH5Juh2y2hF7+Dyv/His4JVU6ZZ6haYTml+Bksucfa4+lKCHnePw1rabf8  
  qHlCfMzP0duODq6sz5EVlwPBLqva+9JrVtcmrAvq/YEcXPTTUs9pisu5BN2BVWy8EINu4t0YPoyqMRdzVmMh"  
}
```

Postman Request to pass Body Parameter



• Sample Response with Encrypted Data

```
{  
  "successStatus": true,  
  "message": "Success",  
  "responseCode": "000",  
  "data":  
    "1aHQwl94wcBE1ISfbx9RwF0erJS7rDrMx+3B0oHzVj4a5lsbhwcScd8T4D2GodkBti858BhRDSB5v5rKll7KuS6xmOaXTXD0mqUgu/dwQKN  
WYSniD9x9XcSRAVZeoVE0AtMgxXJ8Bm36VwqXoCCbhi7Zocl4Eeib2M99UJm+Np0/+H6Vnt9gzW524391nOTn3laUf46lu5riS3Htmwa1gY  
K1nNvL4jTuAaM5GQTCzgWCyzlDgvrZk7k3fFPsJ9+fzAgocJQ364UcnhxbV+m8vUORC0ojUGJzBOQi9tdDuC+Jgl8ZSZwztk7Gyd9gCZV4ih  
ZsNlatoQVns7G5QkboGe47WN+fiMyxS2kzZorGWgRNU7xhV94BYjObQxO9/mSlrHd/WT1uPolSyXhy6sj3s4GW/lxguZ0nFyihDG8ZNRKna  
oY1Wz91d5Tc7fyeWpJiRI9ZohoEgs0rQk/r6zzRty2IEwe6EokKHfdYGOncqqeVldQONCjRvhCJriJjpiEhsGPAdbBB43Zil5/XJA7pPeber1S5P  
G9cOcGG/jWyQReUGg639Z/i4mccpAqRZP9vekOjnaceapzB84e/6VqX929le2IxCzJiHLrIwWLBPHo4h5xSFlq9/Q+ARzzLiyCdNIGqt+ihlWi  
UUWbj0HQJaUKSt/28aV6fxlsOBFluQDXewfljUCy6BJjtT964s05WCWdj4RmcJWXJiAhTX7rJbWEDUIDCJc3Shrli9OKC1Qj/J4W0Mk7Ywc  
ptll62rp74479odNVFfKUUNHpBxbHa7dfBUEP6J9KzTjP1zZ2pNZyVMWPgatVSLVrFlRDMmDd5PIW0nxzunjlBYSDVKq7oTI64Eii4K9dPb3v  
L/o581qpYKCKQKD8ina3RIs3mS+QSBh4Mk9ln708P6RbM1KjoFmO6o8hYySjQPb+BsbyhpjHg0Dt6Sdcz1fhYdG"  
}
```

Sample Response with Decrypted Data for Cash Withdrawal

```
{  
  "status": "000",  
  "merchantStatus": "Success",  
  "statusDescription": "Transaction Success",  
  "txnRefId": "b2284f65268d43a6ab97928b1f3c1539",  
  "merchantRefId": "amit999",  
  "transactionAmount": "150",  
  "aadhaarNo": "XXXXXXXXX0738",  
  "txnDateTime": "16-06-2025 21:22:02",  
  "bankIIN": "608117",  
  "rrn": "515313024709",  
  "npciCode": "00",  
  "npciMessage": "Successful",  
  "availableBalance": "3456.00",  
}
```

Sample Response with Decrypted Data for Cash Withdrawal

```
"pipe": "1"  
"lat": "28.7041",  
"long": "77.1025",  
"udf1": "test1",  
"udf2": "test2",  
"udf3": "test3",  
}
```

Sample Response with Decrypted Data for Mini Statement

```
{  
  "status": "000",  
  "merchantStatus": "Success",  
  "statusDescription": "Transaction Success",  
  "txnRefId": "cc3ab80fe8b540dcb8d8ecf0658198b7",  
  "merchantRefId": "amit999",  
  "transactionAmount": "0",  
  "transactionList": "\"TxnType CR TxnAmount 1000.00 TxnDate 2025-04-28 TxnTime 10:30:00  
  Narration Cash Deposit TxnType DR TxnAmount 500.00 TxnDate 2025-04-28 TxnTime 11:30:00  
  Narration Cash Withdrawal\"",  
  "aadhaarNo": "XXXXXXXXX0738",  
  "txnDateTime": "16-06-2025 21:36:55",  
  "bankIIN": "608117",  
  "rrn": "515313024709",  
  "npciCode": "00",  
  "npciMessage": "Successful",  
  "availableBalance": "3456.00",  
}
```

Sample Response with Decrypted Data for Mini Statement

```
"pipe": "1"  
"lat": "28.7041",  
"long": "77.1025",  
"udf1": "test1",  
"udf2": "test2",  
"udf3": "test3",  
}
```

Sample Response with Decrypted Data for Balance Enquiry

```
{  
  "status": "000",  
  "merchantStatus": "Success",  
  "statusDescription": "Transaction Success",  
  "txnRefId": "66db2095dd7c49e7a44ffe5a00c82219",  
  "merchantRefId": "amit999",  
  "transactionAmount": "0",  
  "aadhaarNo": "XXXXXXXXX0738",  
  "txnDateTime": "16-06-2025 21:35:13",  
  "bankIIN": "608117",  
  "rrn": "515313024709",  
  "npciCode": "00",  
  "npciMessage": "Successful",  
  "availableBalance": "3456.00",  
}
```

Sample Response with Decrypted Data for Balance Enquiry

```
"pipe": "1"  
"lat": "28.7041",  
"long": "77.1025",  
"udf1": "test1",  
"udf2": "test2",  
"udf3": "test3",  
}
```

Sample Response with Decrypted Data for Aadhaar Pay

```
{  
  "status": "000",  
  "merchantStatus": "Success",  
  "statusDescription": "Transaction Success.",  
  "merchantId": "MER8623488",  
  "txnRefId": "bc150ce18e3b48",  
  "merchantRefId": " amit999",  
  "transactionAmount": "200.0000",  
  "transactionList": "",  
  "aadhaarNo": "XXXXXXXXX0720",  
  "txnDateTime": "16-06-2025 21:35:13 ",  
  "bankIIN": "607618",  
  "rrn": "515313024709",  
  "npciCode": "00",  
  "npciMessage": "Transaction Success",  
  "availableBalance": "",  
}
```

Sample Response with Decrypted Data for Aadhaar Pay

```
"pipe": "1"  
"udf1": "test1",  
"udf2": "test2",  
"udf3": "test3",  
}
```

Encryption Logic in C#

```
public static string Encrypt(string PlainText, string EDKey, string IVKey)
{
    byte[] key = Encoding.UTF8.GetBytes(EDKey); byte[] iv = Encoding.UTF8.GetBytes(IVKey);

    string sR = string.Empty;
    try
    {
        byte[] plainBytes = Encoding.UTF8.GetBytes(PlainText);
        GcmBlockCipher cipher = new GcmBlockCipher(new AesFastEngine()); AeadParameters parameters =
        new AeadParameters(new KeyParameter(key), 128, iv, null); cipher.Init(true, parameters);
        byte[] encryptedBytes =
        new byte[cipher.GetOutputSize(plainBytes.Length)];
        Int32 retLen = cipher.ProcessBytes
        (plainBytes, 0, plainBytes.Length, encryptedBytes, 0); cipher.DoFinal(encryptedBytes, retLen);
        sR = Convert.ToBase64String
        (encryptedBytes, Base64FormattingOptions.None);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message); Console.WriteLine(ex.StackTrace);
    }
    return sR;
}
```

Decryption Logic in C#

```
public static string Decrypt(string EncryptedText, string EDKey, string IVKey)
{
    byte[] key = Encoding.UTF8.GetBytes(EDKey); byte[] iv = Encoding.UTF8.GetBytes(IVKey); string sR
    = string.Empty;
    try
    {
        byte[] encryptedBytes = Convert.FromBase64String(EncryptedText);
        GcmBlockCipher cipher = new GcmBlockCipher(new AesFastEngine()); AeadParameters parameters =
        new AeadParameters(new KeyParameter(key), 128, iv, null);
        cipher.Init(false, parameters); byte[] plainBytes =
        new byte[cipher.GetOutputSize(encryptedBytes.Length)];
        Int32 retLen = cipher.ProcessBytes
        (encryptedBytes, 0, encryptedBytes.Length, plainBytes, 0); cipher.DoFinal(plainBytes, retLen);
        sR = Encoding.UTF8.GetString(plainBytes).TrimEnd("\r\n\0".ToCharArray());
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message); Console.WriteLine(ex.StackTrace);
    }

    return sR;
}
```

Encryption Logic in Python

Note: Install pycryptodome & cryptography

```
from Crypto.Cipher import AES
import base64
import logging

# Set up logging
logging.basicConfig(level=logging.ERROR)

# Provided keys
edKey = "8086289ecab4a664c8410a1e79d19090"
ivKey = "7cca987fb6cfeea37022977bde6e62c0"

def encrypt(plainText):
    key = edKey.encode('utf-8')
    iv = ivKey.encode('utf-8')
    encryptedText = None

    try:
        plainBytes = plainText.encode('utf-8')

        if len(key) not in (16, 24, 32):
            raise ValueError("Key length must be 16, 24, or 32 bytes for AES.")
```

Encryption Logic in Python

```
# Note: GCM mode typically expects a 12-byte nonce.
cipher = AES.new(key, AES.MODE_GCM, nonce=iv)

encryptedBytes, tag = cipher.encrypt_and_digest(plainBytes)

# Concatenate the encrypted bytes and the tag
encryptedBytesWithTag = encryptedBytes + tag

# Encode to Base64 to get a string representation
encryptedText = base64.b64encode(encryptedBytesWithTag).decode('utf-8')

except Exception as e:
    logging.error("Encrypt error: {}".format(e))

return encryptedText

# The text you provided
plaintext = "Enter the text which you want to encrypt"

# Encrypt the provided text
encrypted_text = encrypt(plaintext)
print("Encrypted text:")
print(encrypted_text)
```

Decryption Logic in Python

Note: Install pycryptodome & cryptography

```
from Crypto.Cipher import AES
import base64
import logging

# Set up logging
logging.basicConfig(level=logging.ERROR)

# Provided keys
edKey = "8086289ecab4a664c8410a1e79d19090"
ivKey = "7cca987fb6cfeea37022977bde6e62c0"

def decrypt(encryptedText):
    key = edKey.encode('utf-8')
    iv = ivKey.encode('utf-8')
    decryptedText = None

    try:
        encryptedBytesWithTag = base64.b64decode(encryptedText)

        if len(key) not in (16, 24, 32):
            raise ValueError("Key length must be 16, 24, or 32 bytes for AES.")
```

Decryption Logic in Python

```
# Split the encrypted bytes and the tag
encryptedBytes = encryptedBytesWithTag[:-16]
tag = encryptedBytesWithTag[-16:]

cipher = AES.new(key, AES.MODE_GCM, nonce=iv)

# Decrypt and verify the tag
plainBytes = cipher.decrypt_and_verify(encryptedBytes, tag)

decryptedText = plainBytes.decode('utf-8').strip()

except Exception as e:
    logging.error("Decrypt error: {}".format(e))

return decryptedText

# The encrypted text (replace with your actual encrypted string)
encrypted_text = 'Enter the Encrypted text here'

# Decrypt the provided text
decrypted_text = decrypt(encrypted_text)
print("Decrypted text:")
print(decrypted_text)
```

Encryption Logic in php

```
<?php
// Provided keys
$edKey = "8086289ecab4a664c8410a1e79d19090";
$ivKey = "7cca987fb6cfeea37022977bde6e62c0";

// Convert keys to binary format
$key = utf8_encode($edKey);
$iv = utf8_encode($ivKey);

function encrypt($plainText, $key, $iv) {
    $encryptedText = null;

    try {
        // Check key length
        $keyLength = strlen($key);
        if (!in_array($keyLength, [16, 24, 32])) {
            throw new Exception("Key length must be 16, 24, or 32 bytes for AES.");
        }

        // Encryption
        $cipher = "aes-".($keyLength * 8)."-gcm";
```

Encryption Logic in php

```
// Encrypt the plaintext
$tag = "";
$encryptedBytes = openssl_encrypt(
    $plainText,
    $cipher,
    $key,
    OPENSSL_RAW_DATA,
    $iv,
    $tag,
    "",
    16 // Tag length
);

if ($encryptedBytes === false) {
    throw new Exception("Encryption failed: " . openssl_error_string());
}

// Concatenate the encrypted bytes and the tag
$encryptedBytesWithTag = $encryptedBytes . $tag;

// Encode to Base64 to get a string representation
$encryptedText = base64_encode($encryptedBytesWithTag);

} catch (Exception $e) {
    error_log("Encrypt error: " . $e->getMessage());
}

return $encryptedText;
}
0
```

Encryption Logic in php

```
function decrypt($encryptedText, $key, $iv) {
    $decryptedText = null;

    try {
        // Check key length
        $keyLength = strlen($key);
        if (!in_array($keyLength, [16, 24, 32])) {
            throw new Exception("Key length must be 16, 24, or 32 bytes for AES.");
        }

        // Decryption
        $cipher = "aes-".($keyLength * 8)."-gcm";

        // Decode from Base64
        $encryptedBytesWithTag = base64_decode($encryptedText);

        // Split the encrypted bytes and the tag
        $tagLength = 16;
        $encryptedBytes = substr($encryptedBytesWithTag, 0, -$tagLength);
        $tag = substr($encryptedBytesWithTag, -$tagLength);

        // Decrypt the ciphertext
        $decryptedBytes = openssl_decrypt(
            $encryptedBytes,
            $cipher,
            $key,
            OPENSSL_RAW_DATA,
            $iv,
            $tag,
            ""
        );
    }
}
```

Encryption Logic in php

```
if ($decryptedBytes === false) {  
    throw new Exception("Decryption failed: " . openssl_error_string());  
}  
  
$decryptedText = $decryptedBytes;  
  
} catch (Exception $e) {  
    error_log("Decrypt error: " . $e->getMessage());  
}  
  
return $decryptedText;  
}  
  
// The text you provided  
$plaintext = 'Enter the text you want to Encrypt';  
  
// Encrypt the provided text  
$encrypted_text = encrypt($plaintext, $key, $iv);  
echo "Encrypted text:\n";  
echo $encrypted_text;  
  
// Optionally, decrypt to verify  
//$decrypted_text = decrypt($encrypted_text, $key, $iv);  
//echo "\n\nDecrypted text:\n";  
//echo $decrypted_text;  
?>
```

Decryption Logic in php

```
<?php
// Provided keys
$edKey = "8086289ecab4a664c8410a1e79d19090";
$ivKey = "7cca987fb6cfeea37022977bde6e62c0";

// Convert keys to binary format
$key = utf8_encode($edKey);
$iv = utf8_encode($ivKey);

function decrypt($encryptedText, $key, $iv) {
    $decryptedText = null;

    try {
        // Check key length
        $keyLength = strlen($key);
        if (!in_array($keyLength, [16, 24, 32])) {
            throw new Exception("Key length must be 16, 24, or 32 bytes for AES.");
        }

        // Decryption
        $cipher = "aes-".($keyLength * 8)."-gcm";

        // Decode from Base64
        $encryptedBytesWithTag = base64_decode($encryptedText);
```

Decryption Logic in php

```
// Split the encrypted bytes and the tag
$tagLength = 16;
$encryptedBytes = substr($encryptedBytesWithTag, 0, -$tagLength);
$tag = substr($encryptedBytesWithTag, -$tagLength);

// Decrypt the ciphertext
$decryptedBytes = openssl_decrypt(
    $encryptedBytes,
    $cipher,
    $key,
    OPENSSL_RAW_DATA,
    $iv,
    $tag,
    ""
);

if ($decryptedBytes === false) {
    throw new Exception("Decryption failed: " . openssl_error_string());
}

$decryptedText = $decryptedBytes;

} catch (Exception $e) {
    error_log("Decrypt error: " . $e->getMessage());
}

return $decryptedText;
}

12
4
```

Decryption Logic in php

```
// The encrypted text (replace with your actual encrypted string)
$encrypted_text = 'Enter your encrypted text here';

// Decrypt the provided text
$decrypted_text = decrypt($encrypted_text, $key, $iv);
echo "Decrypted text:\n";
echo $decrypted_text;
?>
```

Encryption Logic in Java

```
import javax.crypto.Cipher;
import javax.crypto.spec.GCMParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;

public class AESEncryption {

    private static final String edKey = "8086289ecab4a664c8410a1e79d19090";
    private static final String ivKey = "7cca987fb6cfeea37022977bde6e62c0";

    public static String encrypt(String plainText) {
        String encryptedText = null;
        try {
            byte[] key = edKey.getBytes("UTF-8");
            byte[] iv = ivKey.getBytes("UTF-8");

            if (key.length != 16 && key.length != 24 && key.length != 32) {
                throw new IllegalArgumentException("Key length must be 16, 24, or 32 bytes.");
            }
        }
    }
}
```

Encryption Logic in Java

```
SecretKeySpec keySpec = new SecretKeySpec(key, "AES");

GCMParameterSpec gcmParameterSpec = new GCMParameterSpec(128, iv); // 128-bit authentication tag length

Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");

cipher.init(Cipher.ENCRYPT_MODE, keySpec, gcmParameterSpec);

byte[] encryptedBytes = cipher.doFinal(plainText.getBytes("UTF-8"));

encryptedText = Base64.getEncoder().encodeToString(encryptedBytes);

} catch (Exception e) {
    e.printStackTrace();
    System.err.println("Encrypt error: " + e.getMessage());
}

return encryptedText;
}

public static void main(String[] args) {
    String plaintext = "{\n" +
        "  \"userMobileNo\": \"9422811530\", \n" +
        "  \"merchantRefId\": \"Pronoy\", \n" +
        "  \"amount\": \"1\", \n" +
        "  \"lat\": \"19.251\", \n" +
        "  \"long\": \"72.234\", \n" +
        "  \"udf1\": \"\", \n" +
        "  \"udf2\": \"\", \n" +
        "  \"udf3\": \"\" \n" +
        "}";

    // Encrypt the plaintext
    String encryptedText = encrypt(plaintext);
    System.out.println("Encrypted text:");
    System.out.println(encryptedText);
}
}
```

Decryption Logic in Java

```
import javax.crypto.Cipher;
import javax.crypto.spec.GCMParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;

public class AESDecryption {

    private static final String edKey = "8086289ecab4a664c8410a1e79d19090";
    private static final String ivKey = "7cca987fb6cfeea37022977bde6e62c0";

    public static String decrypt(String encryptedText) {
        String decryptedText = null;
        try {
            byte[] key = edKey.getBytes("UTF-8");
            byte[] iv = ivKey.getBytes("UTF-8");

            if (key.length != 16 && key.length != 24 && key.length != 32) {
                throw new IllegalArgumentException("Key length must be 16, 24, or 32 bytes.");
            }

            SecretKeySpec keySpec = new SecretKeySpec(key, "AES");

            GCMParameterSpec gcmParameterSpec = new GCMParameterSpec(128, iv); // 128-bit authentication tag
length

            Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");

            cipher.init(Cipher.DECRYPT_MODE, keySpec, gcmParameterSpec);

            byte[] encryptedBytes = Base64.getDecoder().decode(encryptedText);

            byte[] decryptedBytes = cipher.doFinal(encryptedBytes);

            decryptedText = new String(decryptedBytes, "UTF-8");

        }
    }
}
```

Decryption Logic in Java

```
catch (Exception e) {  
    e.printStackTrace();  
    System.err.println("Decrypt error: " + e.getMessage());  
}  
  
return decryptedText;  
}  
  
public static void main(String[] args) {  
    // Replace this with the encrypted text obtained from the encryption code  
    String encryptedText =  
    "GnEt0YGucuWgW93AXY9qvrNBhhdP/JxRaOrEsDb9/9i+nCtnqOD1od+F2n34XbOfjpSQ37VPW1bYKJddYCPouxkxg9f  
Y7+sECGCTcEEueaPvqGRCHKmBltmUDvX1IYEclQ3SYuGO/8FzWP8qrl6uud9RkLHTRM0+2Z+kFNSXVT30OvhmaiWQ  
TteMdkQ71ILNXofV6DL0nEAITBMvVfVK0nj4l7H+zYO7S+hROqEPWCwoq6Xbt6+pwZLu0oFtZa9Q9w==";  
  
    // Decrypt the ciphertext  
    String decryptedText = decrypt(encryptedText);  
    System.out.println("Decrypted text:");  
    System.out.println(decryptedText);  
}  
}
```

```
public class EncryptUtils {  
  
    private static final String edKey = "8086289ecab4a664c8410a1e79d19090";  
    private static final String ivKey = "7cca987fb6cfeea37022977bde6e62c0";  
  
    public static String encrypt(String plainText) {  
        String encryptedText = null;  
        try {  
            byte[] key = edKey.getBytes(StandardCharsets.UTF_8);  
            byte[] iv = ivKey.getBytes(StandardCharsets.UTF_8);  
            byte[] plainBytes = plainText.getBytes(StandardCharsets.UTF_8);  
  
            if (key.length != 16 && key.length != 24 && key.length != 32) {  
                throw new IllegalArgumentException("Key length must be 16, 24, or 32 bytes for AES.");  
            }  
  
            // if (iv.length != 12) {  
            //     throw new IllegalArgumentException("IV length must be 12 bytes for GCM mode.");  
            // }  
  
            SecretKeySpec secretKeySpec = new SecretKeySpec(key, "AES");  
            GCMParameterSpec gcmSpec = new GCMParameterSpec(128, iv);  
  
            Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");  
            cipher.init(Cipher.ENCRYPT_MODE, secretKeySpec, gcmSpec);  
  
            byte[] encryptedBytes = cipher.doFinal(plainBytes);  
  
            encryptedText = Base64.encodeToString(encryptedBytes, Base64.NO_WRAP);  
        } catch (Exception e) {  
            e.printStackTrace();  
            return "";  
        }  
  
        return encryptedText;  
    }  
}
```

```
public class DecryptUtils {

    private static final String edKey = "8086289ecab4a664c8410a1e79d19090";
    private static final String ivKey = "7cca987fb6cfeea37022977bde6e62c0";

    public static String decrypt(String encryptedText) {
        String decryptedText = null;

        try {
            byte[] key = edKey.getBytes(StandardCharsets.UTF_8);
            byte[] iv = ivKey.getBytes(StandardCharsets.UTF_8);
            byte[] encryptedBytes = Base64.decode(encryptedText, Base64.NO_WRAP);

            if (key.length != 16 && key.length != 24 && key.length != 32) {
                throw new IllegalArgumentException("Key length must be 16, 24, or 32 bytes for AES.");
            }

            // if (iv.length != 12) {
            //     throw new IllegalArgumentException("IV length must be 12 bytes for GCM mode.");
            // }

            SecretKeySpec secretKeySpec = new SecretKeySpec(key, "AES");
            GCMParameterSpec gcmSpec = new GCMParameterSpec(128, iv);

            Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
            cipher.init(Cipher.DECRYPT_MODE, secretKeySpec, gcmSpec);

            byte[] decryptedBytes = cipher.doFinal(encryptedBytes);

            decryptedText = new String(decryptedBytes, StandardCharsets.UTF_8);
        } catch (Exception e) {
            e.printStackTrace();
        }

        return decryptedText;
    }
}
```

Status Code for API

S. No	Status Code	Description
1	000	Success
2	001	Failed
3	002	Pending
4	003	Validation Failed

Escalation Matrix

Level	Name	Email
Level 1	Pronoy	pronoy.bhattacharya@vidual.in
Level 2	Tekchand	tekchand.sharma@vidual.in

*Thank
you!*